

Herald

Toda alerta llega al destino correcto — sin necesidad de sintaxis de comandos.

Resumen

Herald ingiere eventos del sistema y los distribuye de forma fiable a múltiples canales de notificación para que cada alerta llegue al lugar adecuado. Los suscriptores interactúan en lenguaje natural sencillo; Herald infiere la intención mediante un protocolo de tres niveles (ruta rápida de comandos → inferencia de intención con LLM → retroceso de aclaración).

Descripción breve

Sistema de ingesta de eventos y distribución multicanal de notificaciones. Herald redirige de manera confiable los eventos del sistema a los destinos correctos en distintos canales de mensajería y permite que los suscriptores se comuniquen en lenguaje natural, resolviendo la intención mediante una ruta rápida de comandos, inferencia con LLM y un mecanismo de retroceso para solicitar aclaraciones.

Descripción detallada

Herald es la columna vertebral de notificaciones que garantiza que un evento del sistema llegue efectivamente a donde un ser humano pueda actuar sobre él: esa capa poco vistosa pero crítica en la que fallan silenciosamente la mayoría de los sistemas de alerta caseros. Ingiere eventos y los distribuye de forma fiable a través de múltiples canales de notificación, eliminando los modos de fallo habituales en los que una alerta se pierde, se envía por error a un canal inactivo o queda sepultada bajo el ruido hasta que ya no importa. Pero la entrega confiable es solo la mitad de la historia; la otra mitad es lo que ocurre cuando un usuario quiere responder. Aquí, Herald rechaza el trato habitual en el que los usuarios deben memorizar una sintaxis de comandos rígida para interactuar con un bot de alertas. Los suscriptores simplemente

escriben en lenguaje natural, y Herald interpreta lo que quieren decir mediante un protocolo de tres niveles cuidadosamente diseñado: una ruta rápida que reconoce comandos explícitos al instante, luego la inferencia de intención basada en LLM (mediante Claude Code) para mensajes en formato libre, y, por último, un retroceso de *aclaración* que responde, etiqueta y formula una pregunta cuando la intención es realmente ambigua. Esa escalera de “reconocer → inferir → aclarar” resume toda la filosofía de diseño: el caso común se resuelve de forma instantánea y determinista, el caso flexible se gestiona mediante un modelo, y el caso incierto nunca se resuelve con una suposición ciega que desencadene la acción equivocada.

Herald también modela la participación y la atribución: una variable de entorno con el nombre de usuario del operador (HERALD_<CANAL>_OPERATOR_USERNAME) y un contrato de participación/atribución impulsan los campos created_by/assigned_to y el etiquetado de notificaciones con @, de modo que queda claro quién hizo qué y a quién se notifica. En términos de gobernanza, Herald hereda el Helix Constitution como submódulo co-ubicado y sigue sus normas, y es uno de los primeros consumidores en producción de Docs Chain: su corpus completo de 66 documentos (Markdown→HTML/PDF/DOCX) se procesa mediante transformaciones exec: de Docs Chain y se verifica sin errores. Herald es principalmente una herramienta de Shell/Go con especificaciones estratificadas (sucesión V1→V2→V3→V4) y guías de configuración por canal para operadores de mensajería y despachadores de LLM/agentes.

Por qué lo creamos

Las alertas fallan en silencio: se envían al canal equivocado, se pierden o exigen una sintaxis de comandos rígida que los usuarios no recordarán. Herald se creó para garantizar una distribución fiable y permitir que las personas respondan en lenguaje natural, de modo que las notificaciones sean tanto confiables como fáciles de gestionar.

Por qué es un cambio de juego

Combina dos elementos que normalmente se adquieren como productos separados —el enrutamiento confiable de eventos multicanal y una interfaz en lenguaje natural— en un único sistema donde los operadores simplemente hablan y el software interpreta lo que quieren decir. El mecanismo de aclaración como respaldo es el detalle que lo hace confiable en producción: un

sistema de alertas que prefiere preguntar antes que activarse por error es uno en el que realmente puedes confiar para manejar situaciones reales.

Qué lo hace innovador

- Disciplina de intenciones en tres niveles: ruta rápida de comandos → inferencia LLM → aclaración y consulta.
- Interacción con suscriptores en lenguaje natural (sin sintaxis de comandos que aprender).
- Contrato de atribución de participantes que impulsa `created_by/assigned_to` + etiquetado con @.
- Consumidor real Docs Chain (corpus de 66 documentos, multiformato, verificado).

Desafíos y soluciones

- **Intención ambigua en lenguaje natural:** resuelto con la escalera de reconocimiento/inferencia/aclaración en tres niveles, en lugar de adivinar a ciegas.
- **Distribución confiable:** resuelto con un diseño de ingesta → envío multicanal para que las alertas lleguen al destino correcto.
- **Atribución correcta entre canales:** resuelto con la variable de entorno del nombre de usuario del operador y el contrato de atribución de participantes.
- **Desfase en la documentación:** resuelto al integrar su corpus de documentos mediante Docs Chain con transformaciones verificadas.

Pila tecnológica (por qué y cómo)

- **Go** — lógica central de eventos/dispatch (patrones de lenguaje por organización).
- **Shell** — herramientas para operadores y scripts de configuración.
- **Código Claude (LLM)** — nivel de inferencia de intenciones para mensajes de formato libre.
- **Adaptadores de canales de mensajería** — distribución multicanal de notificaciones.
- **Docs Chain** — pipeline de construcción/verificación de documentación (Markdown→HTML/PDF/DOCX).
- **Submódulo Helix Constitution** — gobernanza y reglas heredadas.